

P2490R3

Zero-overhead exception stacktraces

Published Proposal, 2022-02-28

This version:

<https://wg21.link/P2490>

Issue Tracking:

[GitHub](#)

Author:

[Ed Catmur](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/ecatmur/stacktrace-from-exception/blob/main/paper.md

Table of Contents

1	Overview
1.1	Abstract
1.2	Motivation
1.2.1	Internally handled exceptions
1.2.2	Binary distributed libraries
1.3	Alternatives
1.4	Acknowledgements
1.5	Revision history
2	Suggested syntax
3	Concerns

- 3.1 Implementability
- 3.2 Secrecy
- 3.3 Rethrow
- 3.4 Coroutines
- 3.5 Allocators

4 Implementation experience

- 4.1 Windows (SEH)
- 4.2 Linux (RTTI)

References

Informative References

§ 1. Overview

§ 1.1. Abstract

This paper identifies a concern with part of the **Stacktrace from exception** [\[p2370\]](#) proposal. We suggest an alternative approach and offer experience of potential implementation techniques.

§ 1.2. Motivation

The paper **Stacktrace from exception** [\[p2370\]](#) amply sets out why it is desired to be able to access a stacktrace from exception; that is, when *handling* an exception it should be possible to retrieve a stacktrace from the (most recent) **throw** point of the exception, through the point of handling; and that this should be *transparent* to and not require *cooperation by* or *modification of* throwing code. That paper acknowledges that the cost of taking a stacktrace on *every* exception throw would be prohibitive and proposes a mechanism to disable it via a standard library routine `std::this_thread::set_capture_stacktraces_at_throw` that will set a thread-local flag.

We argue that this approach has sufficient drawbacks as to prevent the paper from achieving the aims of the proposed facility; we propose an alternative interface that leaves implementers the freedom to choose lower-cost implementation strategies, and demonstrate how those strategies can be implemented.

§ 1.2.1. Internally handled exceptions

When an exception is thrown and handled internally by a (possibly third-party) library, under the proposed mechanism the cost of taking a stacktrace will be incurred even if the internal handler does not access it.

Third-party library vendors who use exceptions for control flow may be expected to view the proposed facility negatively; if user code enables it via the proposed mechanism the cost will be considerable even for exceptions that are caught and handled successfully entirely within the third-party library. Thus they are likely to disable the facility at API entry points, both negating the point of the facility for any exceptions that *do* leak out of the third-party library, and interfering with user code that expects it to remain enabled.

§ 1.2.2. Binary distributed libraries

Under the mechanism proposed in [\[p2370\]](#), code would need recompilation and/or relinking to participate in the facility, since the action to check the flag and take a stacktrace occurs at the throw site. It is not unusual that third-party library code is shipped with its own implementations of the exception-raising mechanism, such that it would not participate in the facility until such time as the vendor recompiles and relinks the library, which may not occur for some time.

§ 1.3. Alternatives

We note that C++ exception handling is typically built on top of a lower-level, language-agnostic facility. On Windows this is structured exception handling [\[seh\]](#), while on the Itanium ABI (used by most Unix-style OSes on x64-64) it is the Level I Base ABI [\[itanium\]](#). This lower-level facility uses *two-phase* exception handling; in the first, "search" phase the stack is walked from the throw point to identify a suitable handler, while in the second, "unwind" phase it is walked again from the throw point to the selected handler, this time invoking cleanup (i.e., destructors) along the way. Importantly,

- during the whole of the search phase the stack is still intact, and
- identifying a handler is *dynamic*, calling into a compiler- or library-generated match function.

This suggests a possible alternative mechanism; viz.:

- user code can mark a `catch` block as requiring a stacktrace, either via a special function or via new syntax; and

- on recognising `catch` blocks so marked, the compiler can emit suitable code or data such that *if and when* that `catch` block is selected as a handler for a thrown exception, it takes a stacktrace during search phase, immediately before nominating that `catch` block as a suitable handler; then
- the user code can retrieve that stored stacktrace during exception handling, after stack unwinding.

This approach has several advantages:

- *transparency*: there is no need whatsoever for throwing code to be modified, recompiled or relinked. Indeed, since this mechanism relies solely on changes to the catch site, code using this mechanism may be introduced into existing (perhaps even running) programs without any need for those programs to be recompiled or relinked, as long as that new code has access to any necessary support libraries;
- *zero-cost*: if the search phase does not reach a `catch` block so marked (i.e., if the exception is caught and handled internally) then behavior is entirely unaffected.
- *vendor freedom*: the implementer can implement the facility in whatever way is most efficient and appropriate for the targeted platform.

§ 1.4. Acknowledgements

Thank you especially to Antony Peacock for getting this paper ready for initial submission, and to Mathias Gaunard for inspiration, review and feedback. Thank you also to Jonathan Wakely and to members of BSI IST/5/-/21 (C++) panel for review and feedback.

§ 1.5. Revision history

R0

Initial revision; incorporated informal feedback.

R1

Add `with_stacktrace` proposed syntax; add attribute syntax. Extend discussion of rethrow. Add discussion of fallback implementation, coroutines, and allocators.

R2

Promote attribute syntax.

R3

Clarify motivation

§ 2. Suggested syntax

Note: some more alternative syntaxes are discussed in previous versions of this paper.

We suggest a syntax using an attribute `[[with_stacktrace]]` to the *exception-declaration* of the *handler* requesting exception stacktrace:

```
try {  
    ...  
} catch ([[with_stacktrace]] std::exception& e) {  
    std::cout << e.what() << "\n" << std::stacktrace::from_current_exception(e)  
}
```

This would require one minor grammar change, allowing an *attribute-specifier-seq* to precede the `...` production of *exception-declaration*. By moving the *attribute-specifier-seq* to *handler*, this would in fact be a simplification.

The `[[with_stacktrace]]` attribute would be permitted to appear only on an *exception-declaration* (though see § 3.4 [Coroutines](#)).

Semantically, an exception being handled has an *associated stacktrace*, which the implementation is encouraged to ensure extends at least from its most recent `throw` point (possibly a rethrow, see § 3.3 [Rethrow](#)) to the point where it is caught, only if the *exception-declaration* where it is caught (which may be `...`) has the attribute `[[with_stacktrace]]`; otherwise, the exception does not have an associated stacktrace. The static member function `std::stacktrace::from_current_exception()` (see [\[p2370\]](#)) returns (as `std::stacktrace`) the associated stacktrace of the currently handled exception if one exists, otherwise the return value is unspecified (or possibly empty, or possibly `std::stacktrace::current()`).

Note that the interface for *accessing* the stored stacktrace is the same as in [\[p2370\]](#); it is only the interface for *requesting* that it be stored that is different.

The attribute syntax is in keeping with the nature of this facility as a request to the implementation that can be ignored if unsupported. The syntax makes it easy for users to add to existing code; since unrecognized attributes are ignored it can be unconditionally added and the exception stacktrace retrieved conditional on a feature-test macro, with surrounding code unchanged.

For future direction, this syntax would allow passing parameters via attribute arguments, for example limiting stack depth via a `max_depth` argument to the attribute. It would also make it conceivable to

add further diagnostic information in future (e.g. minidump), in an orthogonal manner by adding more attributes.

A possible disadvantage is that `std::stacktrace::from_current_exception()` would fail at runtime (possibly in an unspecified manner) in case the current exception being handled does not have an associated stacktrace, or if there is no current exception being handled. This does not appear to be a problem in practice with `std::current_exception()`, and can be seen as an advantage if users wish to enable or disable the attribute via the preprocessor conditional on build type.

An implementation of this syntax is presented in [\[branch-attribute\]](#).

§ 3. Concerns

§ 3.1. Implementability

As discussed above, Windows uses SEH [\[seh\]](#), and many Unix-like platforms including Linux use the Itanium ABI [\[itanium\]](#); both of these are zero-cost on the happy path (that is, they do not emit code to be called on entry to or exit from a `try`), and both permit calling arbitrary code during search phase (the former via arbitrary *funclets*, small segments of code with a special calling convention that are used to implement matching and cleanup; the latter via RTTI), before stack unwinding; as such, we suggest that these platforms should implement this facility via a similarly zero-cost approach.

Another exception handling methodology in use is setjmp/longjmp (SJLJ, see [\[llvm\]](#)); this also has two-phase unwinding but registers handlers on a stack at runtime, so could either use a zero-cost approach or choose to store a thread-local flag as in [\[p2370\]](#), instructing the exception mechanism to take a stacktrace at the point of `throw`. This flag would be set to `true` on entry to a `try` block with accompanying `catch` block marked as requiring stacktrace, to `false` on entry to `noexcept` functions and to `try` blocks with accompanying `catch(...)` block not marked as requiring stacktrace, and restored to its previous value during stack unwinding. The overhead of access to thread-local data would be justified since registering handlers requires access to thread-local data (the handler stack) anyway.

Indeed, any platform with two-phase lookup and dynamic search phase (either RTTI- or funclet-based) is suitable for implementation of the proposed mechanism. For platforms that do not fall under this description, a thread-local flag can be used. This would still have the advantage relative to the API suggested in [\[p2370\]](#) that the `capture_stacktraces_at_throw` flag would be hidden from user code, and would be automatically set or restored to the appropriate value according to whether a stacktrace is requested in a particular (dynamic) scope.

Finally, [p0709] suggests a "static" exception mechanism with linear control flow, where exception objects are passed back down the stack alongside return values. Contra [p2370], we believe that this is compatible with exception stacktraces, especially if provision is made from the start; since a new ABI would be required, a per-thread flag could be maintained efficiently without recourse to thread-local storage (e.g. in registers). Code taking the error path would test the flag and, if it is set, push the program counter onto a per-stack array. In addition, since in that proposal the a rethrow (i.e. `throw;`) can only occur in a `catch` block it would be easy to track whether an exception can or cannot escape a particular block and so the value of the flag could be maintained accurately.

§ 3.2. Secrecy

Third-party vendors who view secrecy as a virtue may be tempted to put `catch ( ... )` blocks at API entry points to prevent information on their library internals leaking out. In practice they can achieve much the same end by stripping debug symbols and obfuscating object names, and are likely to do so; meanwhile the same information is available by attaching a debugger.

It has been suggested that the Standard may wish to provide an attribute for users to denote that a stack frame should be omitted from stack traces. We consider this out of scope for this proposal.

§ 3.3. Rethrow

For a *rethrown* exception (using `throw;`, `rethrow_exception`, `throw_with_nested`, etc.) the stacktrace will be truncated from the rethrow point. We could provide mechanisms to alleviate this; for example, we could specify that `throw;` preserves stacktrace (specifically, that the accompanying stacktrace of a rethrown exception begins with the stacktrace captured for the use of its containing `catch`).

However, since `throw;` may be placed within a nested function invocation, the resulting chained stacktrace could either be non-contiguous, indeed self-overlapping (if it restarts from `throw;`) or, if it does not restart, omit important information (the stack from the catch clause to the rethrow point, showing how and why the exception was rethrown). At present, in the light of the complication and potential confusion arising, we choose not to pursue this.

An alternative approach could be to extend `throw_with_nested` to accept a stacktrace and store it in `nested_exception` or a derived class. Since this would be a pure library extension, we are not pursuing it in this paper but leave it open for future direction.

§ 3.4. Coroutines

In several places in the coroutines machinery exceptions are specified as being caught and rethrown, e.g. if the initial suspend throws (before initial *await-resume*), the exception is caught and rethrown to the coroutine caller; from this point onwards, exceptions are caught as if by `...` and `unhandled_exception` is called on the promise. This will result in stacktraces retrieved in the caller being truncated to the rethrow point, and not being available at all to `unhandled_exception`.

The issue of truncation could be addressed by special wording; implementations may be able to use automatic object cleanups (which do not interrupt a stacktrace).

By allowing the `[with_stacktrace]` attribute to appertain to member function declarations as well as *exception-declarations*, it could be applied to a promise type's `unhandled_exception` member function, thereby directing the implementation to make exception stacktrace available to that handler.

§ 3.5. Allocators

Previously, we suggested that some syntaxes could permit the user to supply an allocator; this might be desirable for performance and/or latency. On the other hand, allowing the user to supply an allocator opens the door to abuse (running arbitrary user code during unwinding). Even where the user supplies an allocator, it may not necessarily be invoked at the same time as the stacktrace is captured; an implementation could capture into a separate buffer and allocate the stacktrace exposed to the user at a later time.

Resource usage concerns could be addressed by accepting a `max_depth` argument, as discussed above.

§ 4. Implementation experience

The following proofs of concept and implementations are provided to demonstrate implementability.

§ 4.1. Windows (SEH)

It is well known that the vendor-specific `__try` and `__except` keywords `[try-except]` (present in Visual Studio and compatible compilers) permit arbitrary code to be invoked during search phase, since the *filter-expression* argument to the `__except` keyword is a funclet evaluated during search phase, to an enumeration indicating whether the consequent code block is to be selected as the handler.

We present a proof-of-concept implementation [poc] (32-bit and 64-bit) adapted from an article by Howard Jeng [jeng].

§ 4.2. Linux (RTTI)

Although exception handling on Itanium is also two-phase, the handler selection mechanism is largely hidden from the user. However, there is a workaround involving creating a type whose run-time type information (that is, its typeid) refers to an instance of a user-defined subclass of `std::type_info`. This technique is not particularly widely known, but has been used in several large proprietary code bases to good effect for some time. We present a proof-of-concept implementation [poc] and a fully working branch [branch-attribute] of gcc implementing the suggested `[[with_stacktrace]]` syntax,

§ References

§ Informative References

[BRANCH-ATTRIBUTE]

Branch of GCC implementing proposed attribute syntax. URL: <https://github.com/ecatmur/gcc/tree/with-stacktrace-attribute>

[ITANIUM]

C++ ABI for Itanium: Exception Handling. URL: <https://itanium-cxx-abi.github.io/cxx-abi/abi-eh.html>

[JENG]

The Visual C++ Exception Model. URL: <https://www.gamedev.net/tutorials/programming/general-and-gameplay-programming/the-visual-c-exception-model-r2488/>

[LLVM]

Exception Handling in LLVM. URL: <https://llvm.org/docs/ExceptionHandling.html>

[P0709]

Zero-overhead deterministic exceptions: Throwing values. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0709r4.pdf>

[P2370]

Stacktrace from exception. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2370r2.html>

[POC]

Proof-of-concept implementation.. URL: <https://github.com/ecatmur/stacktrace-from-exception/blob/main/stacktrace-from-exception.cpp>

[SEH]

Structured Exception Handling (C/C++). URL: <https://docs.microsoft.com/en-us/cpp/cpp/structured-exception-handling-c-cpp>

[TRY-EXCEPT]

`try-except` statement. URL: <https://docs.microsoft.com/en-us/cpp/cpp/try-except-statement>